COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate course for games

03

Gate B: Game-mathematics notebook

Project specification and review rubric

Integrate proof, geometry, dynamics, probability, signals and numerical evidence into one reproducible artifact.

PACKAGE 03 / EXPANDED VOLUME 02 / SEPTEMBER 28, 2026

Self-study: complete the derivations, implementation, testing and review.

Gate B — Game-mathematics notebook

Original checkpoint requirement

After Volume 2. Produce a versioned notebook or scripts plus written derivations. Required artifacts are a logic/invariant proof, vector and matrix operations, a transform-chain example, a one-dimensional dynamics model, a sampling/aliasing demonstration and a small statistically analyzed experiment.

Tests must include a known transform, a zero-vector edge case, an analytic motion comparison, a timestep refinement study and a probability calculation checked by simulation. Show units and numerical tolerances. Do not normalize a zero vector silently or treat a random experiment as proof. Complete and log the required source problem sets.

The oral check supplies a changed coordinate convention, a new timestep and a misleading average. The learner must explain what changes and what remains invariant. A screenshot of a plot without derivation or reproducible data is insufficient.

Expanded submission contract

Build one versioned notebook or a set of scripts with a written report. Jupyter is optional: ordinary Markdown, Python and CSV are sufficient. Integrate the work already produced in Modules 06-10. Name the question, model, implementation and evidence for every section. Running the reference files unchanged is a demonstration, not the learner's submission.

The notebook must contain six sections: a logic/invariant argument; tested geometry and a transform chain; analytic and numerical dynamics; probability and an experiment; sampling/filtering; and an optimization/error-budget review. The six sections make the original checkpoint concrete; they are not six thesis-scale projects. Keep the dataset and state space small enough to reproduce from a clean folder.

Suggested folder layout

```
my_gate_b/
  README.md
  notebook.md
  assumptions.md
  source_problem_log.md
  src/
  tests/
  inputs/
  evidence/
  reproduce.py
```

Use the template under capstone/student/ as a starting document. Your code should perform the actual calculations and tests, not paste reference CSV outputs into a report. Keep commands and input configuration under version control. Avoid committing personal information or unnecessary generated binary files.

Section 1 — Logic, counting and a specification

Formalize at least five requirements over explicit domains. Include a complete access-rule truth table, a proof by induction, and one loop invariant with initialization, preservation, exit use and termination measure. Supply three counterexamples to false claims. Define what every edge in a startup graph means and include a cycle that invalidates an ordinary startup order.

The reviewer should be able to challenge the domain and obtain a precise answer. A universal claim about all live entities does not imply that a live entity exists. A list of passing Boolean examples is not a proof over an unbounded domain. The supplied checker can test an implementation but cannot accept a proof on your behalf.

Section 2 — Geometry and coordinate conventions

Declare units, handedness, row/column convention, axis directions, multiplication order and the roles of w=0 and w=1. Implement/test vector operations and matrix composition. Demonstrate a parent transform, a child offset and a camera inverse. Include a unit-quaternion rotation and explain why q and -q describe the same orientation under the declared rotation formula.

A useful known case is parent T(10,0,0)*Rz(pi/2), child T(2,0,0), applied to the child origin with column vectors. The result is (10,2,0,1). Translation must not affect a direction with w=0. Include identity, inverse round-trip, zero-vector rejection, mismatched dimensions, singular solve and a deliberately reversed composition-order failure. A camera view matrix is not simply its local-to-world matrix.

Section 3 — Dynamics and numerical error

First state the model: a one-dimensional particle with constant acceleration, plus a linear spring with mass m, stiffness k and damping c. Record initial conditions and units. Derive x(t)=x0+v0*t+0.5*a*t^2 and v(t)=v0+a*t. Explain omitted rotation, collisions and other forces. The model's omissions are distinct from numerical error.

Compare explicit and semi-implicit Euler over the **same final time** at no fewer than three timesteps. For the undamped case use the analytic oscillator reference where appropriate; examine position and energy, including a deliberately unstable case. The supplied numerical example uses m=1, k=100, c=0, x0=1, v0=0, T=2. For this specific linear oscillator, the semi-implicit bounded regime is 0<omega*h<2 with omega=sqrt(k/m); this is not a stability theorem for all damped/nonlinear systems.

Report a refinement table, not only a smooth plot. Explain which errors are expected to shrink and what remains when the physical model is wrong. Keep a nonfinite or diverging trajectory visible as a failed run; do not drop it solely because it spoils a preferred method's graph.

Section 4 — Probability and an experiment

Compute at least one probability analytically and check it with a controlled simulation. Distinguish P(alert|bug) from P(bug|alert). For a categorical reward model, record weights, category order, generator, seed and sample count. Compare multiple counts/seeds without treating finite random frequencies as mathematical proof. Reusing one seed across different sample counts can share prefixes and does not create independent replications across those counts.

Then measure two correct implementations of one small workload on the same machine. The supplied optional harness compares linear list membership with set construction plus membership. Its setup cost is included for the set method. Preserve input generation rules, run order, warmups, raw observations, validation and all retained/excluded runs. Use medians and at least one declared empirical percentile. A percentile definition is part of the report.

A conditional Student-t interval for the mean paired difference is acceptable only with a discussion of its assumptions and limitations. Consecutive runs may share scheduling, thermal and cache conditions. A tiny p-value or narrow interval cannot establish broad practical benefit, normality, independent trials or portability by itself. It is acceptable to conclude that the evidence is too noisy for a confident preference.

Section 5 — Sampling and filtering

Produce a sampling example with an analytically known alias relationship. For zero-phase cosines sampled at fs, frequencies f and fs-f have identical samples at integer n; sine has a sign/phase distinction that must be stated. Demonstrate downsampling a small checker or alternating sequence with and without a prefilter. State whether the filter is an illustration or a design that actually meets a band-rejection requirement.

Implement full zero-extended convolution and predict an impulse, constant and step response. Use an asymmetric kernel so a correlation implementation does not accidentally pass symmetric tests. Report boundary behavior and delay. Filtering after aliases have overlapped cannot generally recover the unique original signal.

Section 6 — Optimization and an error budget

Derive one scalar and one multivariable gradient. Check them numerically over several perturbation sizes. Solve a stated quadratic with a finite iteration budget and test a stable, oscillating and divergent step size. A returned point with iterations exhausted must be marked nonconverged unless it actually satisfies the stopping rule.

Provide a table allocating approximation error among model, discretization, finite precision and iteration termination where applicable. Bounds must share output units before they can be combined. Summing absolute bounds is conservative; root-sum-square requires a justified statistical model. A stationary gradient is not automatically a global minimum of an arbitrary function.

Minimum evidence and review checks

Area	Essential evidence	Required adverse case
Logic	Domain, truth table, induction and loop proof	False claim with a counterexample
Geometry	Known transform and camera inverse	Zero normalization and singular solve
Dynamics	Analytic comparison, fixed-horizon refinement	Unstable or nonfinite trajectory
Probability	Analytic model plus seeded counts	Zero-probability conditioning rejected
Experiment	Raw times, correct outputs, declared trial unit	Small workload or scheduling noise
Signals	Sampling relation and predicted convolution	Asymmetric kernel and edge behavior
Optimization	Gradient check, history and stopping status	Iteration-limit nonconvergence
Reproduction	Clean command, source and environment	Wrong input yields a readable failure

At least 20 learner-written automated checks should cover the relevant contracts and integration, with a clear requirement mapping. This is an expanded implementation target, not a claim that 20 tests exhaust correctness. The supplied 230 reference checks may inform your design but do not count as learner-authored work. Include at least two worked derivations/proofs from each module and the 75 problem attempts/corrections or a reviewer-approved equivalent source set.

Numerical tolerance policy

The small supplied code checks usually compare outputs with absolute tolerance 1e-9 and relative tolerance 1e-8; individual comparisons may be exact or use a specific tolerance. Those are educational test settings, not universal error budgets. Absolute tolerance matters around zero, and relative tolerance scales with magnitude. Choose an application tolerance before inspecting a desired result, explain it, and report actual errors. Rounding printed output is not a repair for an unstable algorithm.

Reproduction and evidence categories

From a clean extraction, confirm the interpreter and run the learner's README commands without editing paths. Keep generated output separate from input. Record which results are analytic, simulated, synthetic or physically measured. The bundled capstone/reference_evidence/numerics files are model outputs. data/synthetic_frame_times.csv and synthetic_regression.csv are invented examples. capstone/reference_evidence/measured_benchmark contains actual measurements from the recorded Linux authoring environment; those values are not performance guarantees for the learner's computer.

The reference generator is rerunnable and refuses to overwrite an existing directory:

```
python capstone/reference/build_evidence.py --out outputs/new_reference
python run_benchmark.py --out outputs/my_benchmark --pairs 30
```

The first command regenerates numerical/model evidence. The second takes new measurements, so its timings should not be expected to match byte-for-byte. A clean source tree is not proof that all external machine conditions were controlled.

Review milestones

Start with contracts and one hand-calculated oracle. Next complete the geometry and constant-acceleration references. Then add the spring, sampling and probability experiments. Finally conduct the actual benchmark, assemble the report and ask another person to reproduce it. Preserve corrections and negative results instead of changing the original question after seeing outcomes.

The oral review should change a coordinate convention, timestep and interpretation of a misleading average, as the original gate requires. It should also ask for one proof step, one seed-dependence explanation and a live modification of a numerical function. The learner must identify what was written independently, what was supplied, and what assistance was used.

Final rubric

Criterion	Points	What earns the points
Correctness and requirements	30	Explicit contracts and correct results in the declared scope
Tests and verification	25	Analytic or independent oracles, edge cases and reviewed proofs
Reasoning and tradeoffs	15	Assumptions, alternatives and limits clearly defended
Quantitative analysis	10	Error, distributions and measured/model distinctions
Reproducibility	10	Clean commands, raw data, pinned configuration and readable failures

Criterion	Points	What earns the points
Communication and provenance	10	Honest report, source log, attribution and change record

Suggested pass is 80/100 plus all essential checks and an independent reproduction/review. Do not compensate for a false correctness claim with attractive plots. No independent student defense has already been completed by the supplied files.